📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی (Main Memory)

📝 بخش ۱: مفاهیم پایه و سخت‌افزار (Background)

۱. مقدمه: چرا حافظه مهم است؟

پردازنده (CPU) فقط می‌تواند مستقیماً به دو جا دسترسی داشته باشد:

1. **ثبات‌های داخلی (Registers)**
2. **حافظه اصلی (Main Memory)**

بنابراین، هر برنامه‌ای برای اجرا شدن **باید** ابتدا وارد حافظه اصلی شود. (دیسک سخت خیلی کند است و CPU مستقیم با آن حرف نمی‌زند).

۲. حفاظت سخت‌افزاری (Hardware Protection) 🛡️

چطور مطمئن شویم که یک برنامه کاربر (User Process) به اشتباه روی حافظه سیستم‌عامل یا برنامه‌های دیگر نمی‌نویسد؟ با استفاده از دو ثبات (Register) مخصوص:

الف) ثبات پایه (Base Register)

- شروع آدرس قانونی برنامه را نگه می‌دارد. (مثلاً ۳۰۰,۰۴۰)

ب) ثبات حد (Limit Register)

اندازه (طول) محدوده مجاز برنامه را نگه می‌دارد. (مثلاً ۱۲۰,۹۰۰)

نحوه کار (خیلی مهم): وقتی CPU آدرسی تولید می‌کند:

1. چک می‌کند آدرس $\geq$ **Base** باشد.
2. چک می‌کند آدرس $<$ **Base + Limit** باشد.
- اگر هر کدام نقض شود $\leftarrow$ **خطا (Trap)** به سیستم‌عامل فرستاده می‌شود (دسترسی غیرمجاز).

- *نکته:* تغییر این دو ثبات یک **دستور ممتاز (Privileged Instruction)** است و فقط سیستم‌عامل می‌تواند آن‌ها را تغییر دهد.

۳. پیوند آدرس (Address Binding) ⛓️

برنامه‌ها روی دیسک هستند و آدرس‌هایشان نمادین است. برای اجرا باید به آدرس‌های حافظه واقعی تبدیل شوند. این "اتصال" در ۳ زمان مختلف ممکن است رخ دهد:

۱. زمان کامپایل (Compile Time)

- اگر دقیقاً بدانیم برنامه کجای حافظه می‌رود.
- تولید **کد مطلق (Absolute Code)** می‌کند.
- *عیب:* اگر بخواهیم جای برنامه را عوض کنیم، باید دوباره کامپایل شود. (مثل سیستم‌های قدیمی MS-DOS).

۲. زمان بارگذاری (Load Time)

- اگر جای برنامه در زمان کامپایل معلوم نباشد.
- کامپایلر **کد جابجاپذیر (Relocatable Code)** تولید می‌کند.
- آدرس نهایی وقتی برنامه بارگذاری می‌شود (Start) تعیین می‌شود.

۳. زمان اجرا (Execution Time) 💥

- اگر برنامه بتواند در حین اجرا از یک جای حافظه به جای دیگر جابجا شود.
- آدرس‌دهی تا آخرین لحظه (زمان اجرا) به تأخیر می‌افتد.
- **نیاز به سخت‌افزار خاص (MMU) دارد.** (اکثر سیستم‌عامل‌های مدرن از این روش استفاده می‌کنند).

۴. آدرس منطقی در برابر فیزیکی (Logical vs. Physical) 🤯

این مهم‌ترین مفهوم این فصل است!

- **آدرس منطقی (Logical Address):**
  - آدرسی که **CPU** تولید می‌کند.
  - به آن **آدرس مجازی (Virtual Address)** هم می‌گویند.
  - برنامه‌نویس و برنامه فقط این آدرس را می‌بینند.
- **آدرس فیزیکی (Physical Address):**

- آدرسی که **واحد حافظه** (روی برد اصلی) می‌بیند.
- مکان واقعی داده در سخت‌افزار.

واحد مدیریت حافظه (MMU) چیست؟

یک سخت‌افزار است که وظیفه تبدیل **آدرس منطقی** $\rightarrow$ **آدرس فیزیکی** را در زمان اجرا بر عهده دارد.

روش ساده (Relocation Register):

- MMU یک **ثبات جابجایی (Relocation Register)** دارد.
- هر آدرسی که CPU تولید کرد را با مقدار این ثبات **جمع** می‌کند تا آدرس فیزیکی ساخته شود.
  - $Physical = Logical + Relocation$
- *مثال:* اگر ثبات جابجایی ۱۴۰۰۰ باشد و برنامه آدرس ۳۴۶ را بخواهد، آدرس فیزیکی ۱۴۳۴۶ می‌شود. برنامه هرگز عدد ۱۴۳۴۶ را نمی‌بیند!

۵. بارگذاری پویا (Dynamic Loading) 🎁

- **هدف:** صرفه‌جویی در حافظه.
- **روش:** کل برنامه یکجا بارگذاری نمی‌شود. هر تابع (Routine) فقط زمانی که **فراخوانی (Call)** شود، به حافظه می‌آید.
- **مزیت:** کدهای کم‌استفاده (مثل کدهای مدیریت خطا) حافظه را اشغال نمی‌کنند.
- **نکته:** نیاز به پشتیبانی خاصی از سیستم‌عامل ندارد (خود برنامه‌نویس یا کتابخانه‌ها انجام می‌دهند).

۶. اتصال پویا و کتابخانه‌های مشترک (Dynamic Linking & Shared Libraries) 🤝

- **اتصال ایستا (Static):** کدهای کتابخانه (مثل printf در C) داخل فایل اجرایی شما کپی می‌شوند. فایل حجیم می‌شود.
- **اتصال پویا (Dynamic):** کد کتابخانه داخل فایل اجرایی نمی‌آید. فقط یک "لینک" (Stub) گذاشته می‌شود.
  - در زمان اجرا، سیستم‌عامل چک می‌کند آیا آن کتابخانه در حافظه هست؟ اگر بود، وصل می‌شود.
- **کتابخانه‌های مشترک (Shared Libraries / DLL):**

- فقط **یک نسخه** از کتابخانه در حافظه بار می‌شود و همه برنامه‌ها از همان استفاده می‌کنند.
- *مثال:* فایل‌های .dll در ویندوز.
- **نیاز به پشتیبانی سیستم‌عامل دارد.**

📊 جدول طلایی مقایسه (جمع‌بندی)

مفهوم	توضیح کوتاه	نکته کلیدی
Base / Limit	ثبات‌های محافظت حافظه.	فقط OS می‌تواند تغییرشان دهد.
MMU	سخت‌افزار تبدیل آدرس.	آدرس منطقی را به فیزیکی تبدیل می‌کند.
آدرس منطقی	تولید شده توسط CPU.	در روش "زمان اجرا" با فیزیکی فرق دارد.
بارگذاری پویا	بارگذاری تابع هنگام فراخوانی.	نیاز به OS ندارد (کتابخانه‌ای است).
اتصال پویا	اتصال کتابخانه هنگام اجرا.	نیاز به OS دارد (اشتراک‌گذاری کد).